



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Algorithms and Data Structures 1

Semestral project

Road Traffic Accident Data Processing

Denis Úradník

Contents

1	Application design	2
1.1	UML	2
2	Data structures used	4
2.1	Reused data structures	5
2.2	Structure 1: DataFrame (the primary sequence)	5
2.3	Structure 2: Data_Structure (year-month-day hierarchy)	5
2.4	Structure 3: Hash_Map_Data_Structure (the table)	6
2.5	Structure 4: data_sorter (the sorting object)	6
3	Levels	6
3.1	Level 1	7
3.2	Level 2	9
3.3	Level 3	10
3.4	Level 4	12
4	User guide	12
4.1	Command overview	13
4.2	Filter flags	14
4.3	Typical scenario	15
4.4	Exiting	15
5	Programmer's guide	15
5.1	Adding a new predicate	15
5.2	Adding a new processor	15
5.3	Adding a new comparator for sorting	16
5.4	Adding a new type of hierarchy traversal	16
5.5	Extending the Accident record	16

1 Application design

The application processes road traffic accident data from the United Kingdom (source gov.uk) across four functional levels: sequential record filtering, hierarchical navigation by date, fast lookup through a hash table, and universal sorting using comparators. All levels operate over a single primary data structure that is loaded only once. Higher levels hold only pointers to the records of the first level, so no data is duplicated in the application.

The architecture is split into three layers:

1. **Domain layer** (`lib/structs/`): defines the `Accident` record and a set of enumerated (enum class) types for the categorical fields (weather, junction, severity, day of week, pedestrian crossing). The record is designed compactly (23 bytes, primarily `uint8_t`, `uint16_t`, `std::chrono::year_month_day`, `std::chrono::minutes`) so that the primary sequence fits into memory even for millions of records.
2. **Data structure layer** (`lib/data_structre/`, `libdf/data_handlers/`): contains custom structures built on top of the `libds` library from the exercises. Each structure addresses one level of the semestral project, and all of them operate over the same data:
 - `DataFrame<T>`: sequential container for level 1 (owner of the records),
 - `Data_Structure + HierarchyIterator`: multi-way year-month-day hierarchy for level 2,
 - `Hash_Map_Data_Structure + AccidentBucket`: hash table keyed by date for level 3,
 - `data_sorter<T>`: standalone sorting object with an internal `MergeSort` for level 4.
3. **CLI layer** (`lib/cli/`, `libdf/cli/`): an interactive command-line application. `CommandLineInterface` reads input from `stdin`, treats the first word as the command keyword and the rest as arguments. These are delegated to `Cli_Command_Logic`, which performs the actual work (parsing filter flags, holding the predicate, running the universal algorithm, navigating the hierarchy, looking up in the table, sorting). The helper class `Cli_Helper_Printer` groups all user-facing text (help, flag descriptions, enum descriptions) and provides the `procPrint` processor for formatted record output.

The universal algorithm `DataFrame::execute(begin, end, predicate, processor)` is a single shared core that is invoked from all four levels. When filtering over a sequence (L1) it receives its iterators; over the hierarchy (L2) it receives a `HierarchyIterator`; when printing a sorted structure (L4) it receives the iterators of a temporary `std::vector`. Both the predicate and the processor are passed as `std::function`, so a single algorithm handles all combinations without any change to its body.

1.1 UML

For readability the class diagram is split into three logically separate views. The structures used from the `libds` library are shown in the diagrams only as class headers, in line with the project rules. The full PlantUML sources are located in the `doc/semestral_project/img/` folder.

Domain model: the `Accident` record and the enumerated types:

6UI0004 Algorithms and Data Structures 1

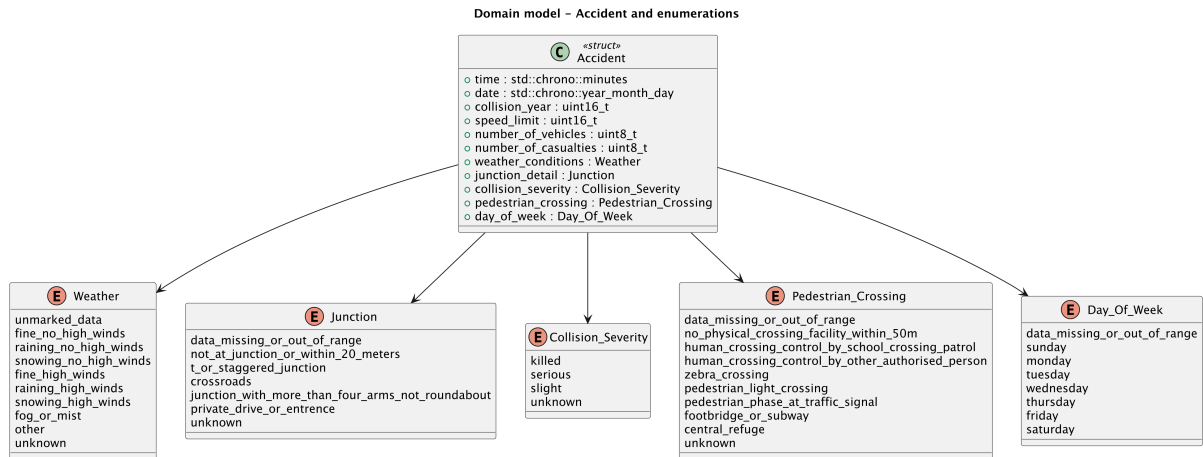


Figure 1: Domain model

Data structures: DataFrame, Data_Structure (with the dependencies Node, HierarchyIterator), Hash_Map_Data_Structure (with AccidentBucket) and the standalone data_sorter. The grey blocks on the right represent the classes reused from libds:

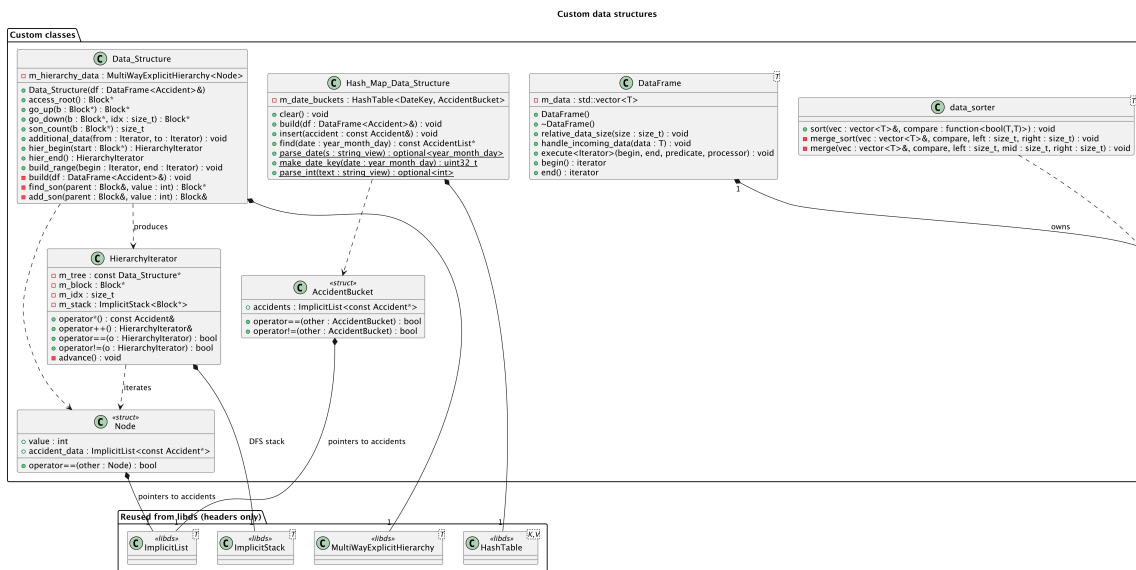


Figure 2: Data structures

CLI and orchestration: Terminal, CSV_Loader, Cli_Helper_Printer, Cli_Command_Logic and CommandLineInterface:

6UI0004 Algorithms and Data Structures 1

CLI and orchestration

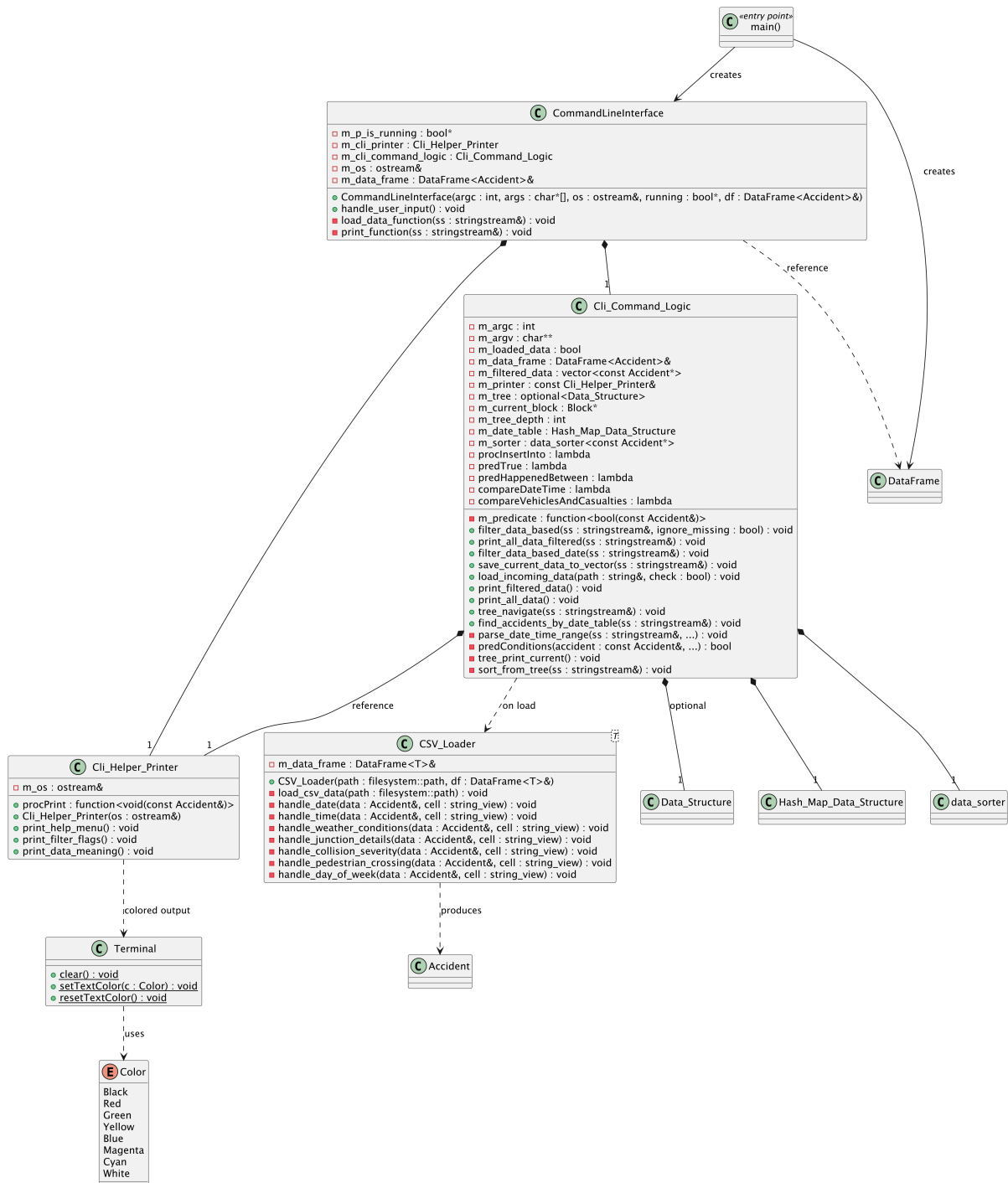


Figure 3: CLI layer

2 Data structures used

The application uses custom classes that each encapsulate one specific responsibility (filtering, hierarchical navigation, hash lookup, sorting), and internally they rest on structures reused from the libds library built during the exercises. The road accident records are physically stored

only once, in `DataFrame<Accident>`; all other structures hold exclusively pointers (`const Accident*`) to those records. This satisfies the assignment's no-duplication requirement.

2.1 Reused data structures

The implementation of the following data structures is reused from the exercises (the `libds` library):

- `ds::amt::MultiWayExplicitHierarchy<T>`: an explicit multi-way hierarchy, used as the storage for the year/month/day tree in `Data_Structure`.
- `ds::amt::MultiWayExplicitHierarchyBlock<T>`: a block (node) of the hierarchy.
- `ds::adt::ImplicitList<T>`: a sequence with implicit ordering over `ImplicitSequence`. Used for the pointer list in the hierarchy leaves and in the hash table cells.
- `ds::adt::ImplicitStack<T>`: a stack over an implicit sequence, used in `HierarchyIterator` during the DFS traversal of a subtree.
- `ds::adt::HashTable<K, V>`: a table with scattered records, pointers to accidents addressed by the hash of the date.

2.2 Structure 1: `DataFrame` (the primary sequence)

`DataFrame<T>` is a thin template wrapper over `std::vector<T>` with the method `execute(begin, end, predicate, processor)`. It serves as the owner of all loaded records. Using `std::vector` is explicitly permitted by the project rules for level 1 (the only structure outside `libds` into which data is loaded).

Suitability: a direct (vector) sequence is well suited for the initial load and sequential filtering: amortized $O(1)$ `push_back` while building with pre-allocated capacity (`reserve`), $O(n)$ traversal via `begin()`/`end()`, good memory locality, and compatibility with the universal algorithm through iterators. Other structures (list, hierarchy, table) would require more overhead here without any benefit.

2.3 Structure 2: `Data_Structure` (year–month–day hierarchy)

`Data_Structure` is a wrapper over `MultiWayExplicitHierarchy<Node>` from `libds`. The hierarchy has four levels: root -> year -> month -> day. In each leaf (at the day level) there is an `ImplicitList<const Accident*>` with pointers to records from the `DataFrame`. The class provides navigation operations (`access_root`, `go_up`, `go_down`, `son_count`) and the subtree iterator `HierarchyIterator`.

Suitability: level 2 requires the primary sequence to be logically partitioned by the date components. A multi-way hierarchy exactly reflects the relational year -> month -> day relationship and allows navigation across the individual levels in $O(1)$ (when moving to a parent). With the built-in row order of the CSV (chronological) this also achieves linear loading: after a record is read, only the last sons at the end of the active branch are checked.

HierarchyIterator is a forward iterator (`std::forward_iterator_tag`) with the standard type aliases (`value_type`, `reference`, `pointer`, `difference_type`). Internally it maintains a stack (`ImplicitStack<Block*>`) of blocks still to be processed. This design satisfies the requirement that the iterator be a standalone object with its own methods, not merely a local pointer to a node.

2.4 Structure 3: Hash_Map_Data_Structure (the table)

Hash_Map_Data_Structure is a wrapper over `ds::adt::HashTable<DateKey, AccidentBucket>`, where `DateKey = uint32_t` is a date encoded as `YYYY·10000 + MM·100 + DD` and `AccidentBucket` is a wrapper over `ImplicitList<const Accident*>`. The cell wrapper is needed because `HashTable` in `libds` requires `operator==/operator!=` on the value type; `ImplicitList` does not provide them.

Suitability: level 3 requires efficient lookup of records for a given date. A hash table provides average $O(1)$ complexity for both insert and find, which is the best possible for this kind of query. A sequential sorted table would require $O(\log n)$ via binary search, as would a BST/Treap. For a few years and a million records, a hash table is therefore a sensible choice. Duplicates are not handled by collisions in the table (for different dates the hash function would scatter them normally) but at the cell level: all accidents with the same date live in a single `AccidentList`. This way no record is lost on insertion (requirement L3).

2.5 Structure 4: data_sorter (the sorting object)

`data_sorter<T>` is a standalone template sorting object in the dedicated header file `lib/data_structre/sorter/data_sorter.hpp` that does not depend on any other part of the project (only `<functional>` and `<vector>`). It implements a recursive top-down Merge-Sort with a deterministic $O(n \log n)$ complexity even in the worst case. The public API consists of a single method `sort(std::vector<T>& vec, std::function<bool(const T&, const T&> compare)`.

Suitability: assignment L4 explicitly excludes selection sort, insertion sort, bubble sort and their variants, and requires a complexity better than $O(n^2)$. MergeSort satisfies both points; in addition it is stable (it preserves the input order for elements with the same key), which is useful with the `compareDateTime` comparator, where accidents at the same moment may share the same key. The comparator is passed as a `std::function`, so the sorting criterion can be changed via lambda functions without any modification to the algorithm body.

In the project the unit being sorted is `const Accident*`, so only eight-byte pointers are moved during sorting, not whole records.

3 Levels

3.1 Level 1

The universal algorithm is implemented by the method `DataFrame<T>::execute(Iterator begin, Iterator end, const std::function<bool(const T)>& predicate, const std::function<void(const T)>& processor)`. The algorithm traverses the range `[begin, end]`, evaluates predicate for each element, and if it returns true, calls processor on it. Neither the predicate nor the processor is fixed anywhere in the algorithm body; the inputs are iterators (not a concrete structure): a dedicated object, a template `Iterator`, the predicate and the processor via lambda functions, and usability for printing as well.

In the `Cli_Command_Logic` class three predicates and two processors are defined (all lambda functions):

- `predTrue`: always true.
- `predHappenedBetween`: the date and time fall within the given interval.
- `predConditions`: a combined check of optional parameters: year, speed limit, number of vehicles (min, max), number of casualties (min, max), weather, junction, severity, crossing, day of week.
- `procInsertInto`: inserts a pointer to the record into a `std::vector<const Accident*>` (the filtered structure).
- `procPrint`: prints the record to standard output in a formatted form (from `Cli_Helper_Printer`).

The UML activity diagram of the universal algorithm is in `doc/semestral_project/img/`:

Universal algorithm: DataFrame<T>::execute(begin, end, predicate, processor)

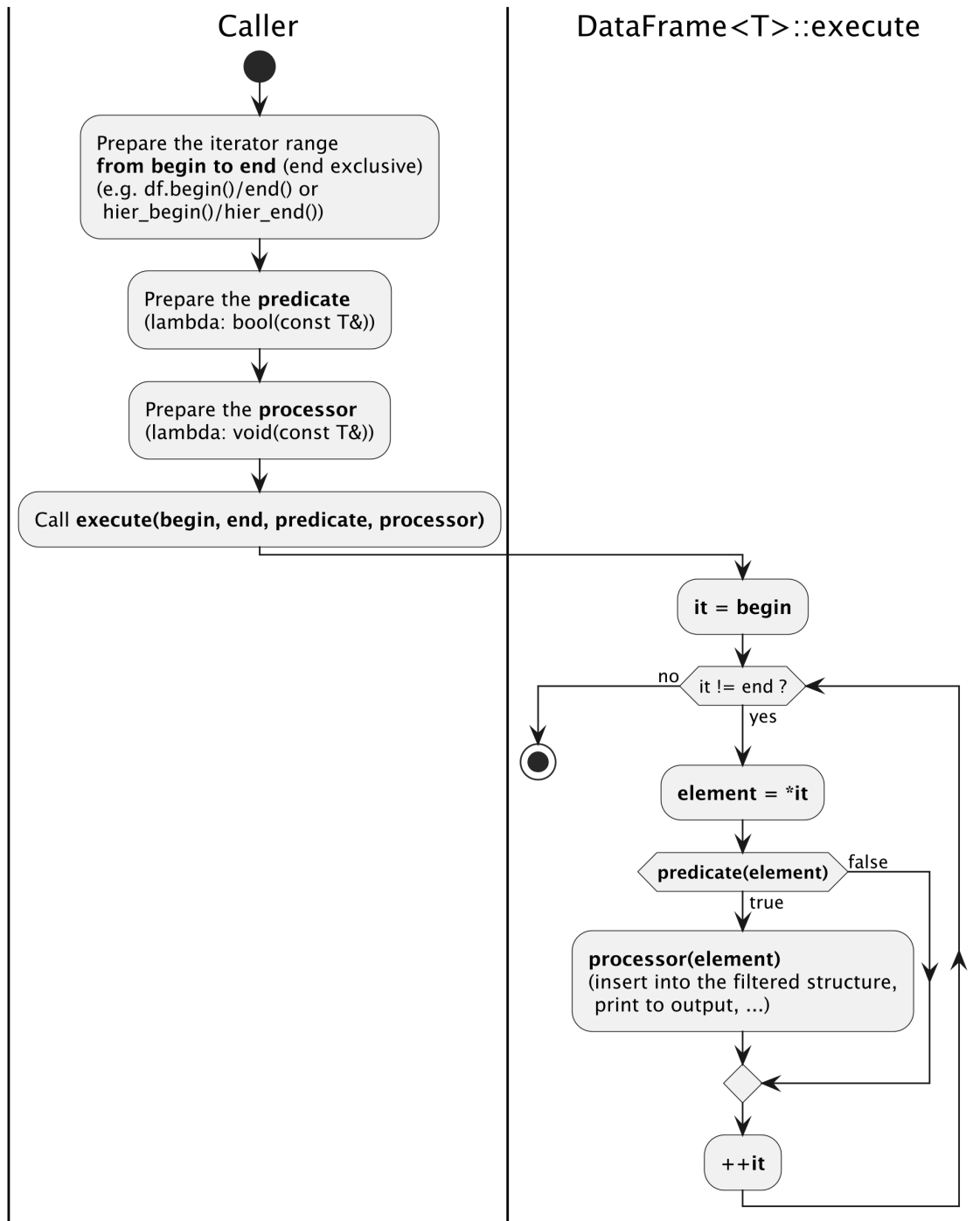


Figure 4: Activity diagram of DataFrame::execute

Edits to the input CSV files. The original gov.uk files were not modified in terms of content (neither the number of rows nor columns changed). The only edit was converting the character set from Windows-1252 to UTF-8. No accident values were lost. The columns loaded are: date, time, collision_year, number_of_vehicles,

6UI0004 Algorithms and Data Structures 1

number_of_casualties, weather_conditions, junction_detail, collision_severity, speed_limit, pedestrian_crossing, day_of_week: CSV_Loader finds them by the header row, so the column order in the input file does not matter.

3.2 Level 2

How the hierarchy is loaded. Data_Structure is built from the primary sequence DataFrame<Accident> in the build() method. It is assumed that the records in DataFrame are chronologically ordered (preserved from the CSV; the UK DTF dataset is delivered in this order). Thanks to this, it is not necessary to search for an existing year/month/day node from the root on every insertion: a pointer to the current year/month/day block is kept, and:

1. For a new record, the date components are compared with the current blocks.
2. If the year/month/day matches, the pointer is inserted directly into the leaf (day).
3. If the year and month match but the day differs, a new day node is attached under the current month. Analogously for month and year.
4. The const Accident* pointer is inserted into the ImplicitList of the day node in $O(1)$.

This makes the whole load run in $O(n)$ complexity with a constant number of checks per record, and each CSV row is opened only once (in CSV_Loader).

Visualization of the hierarchy:

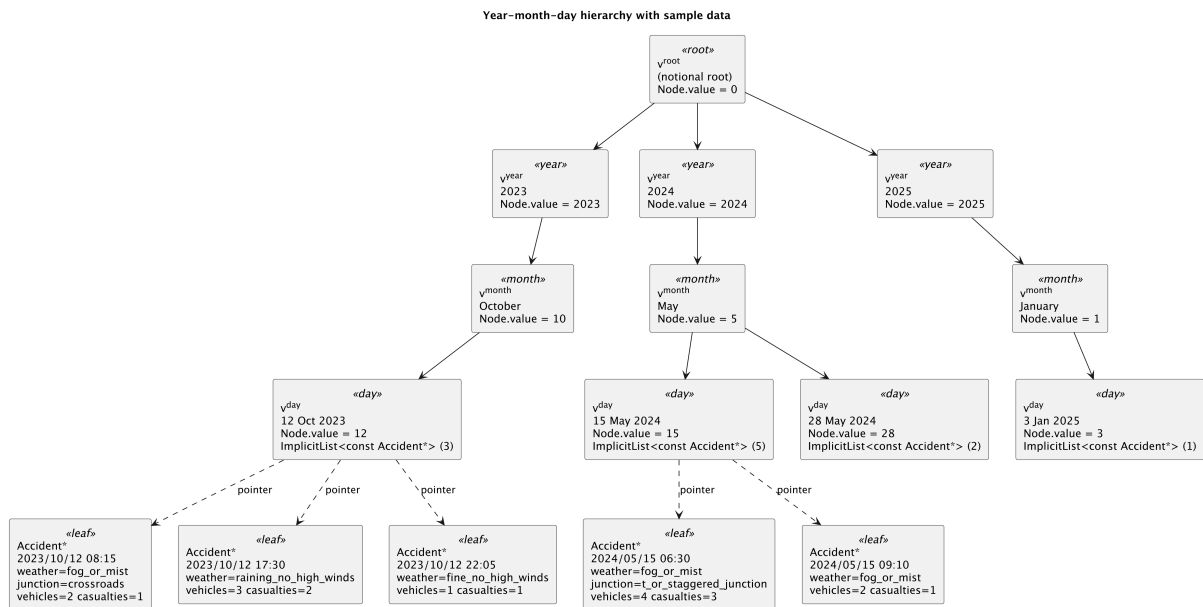


Figure 5: Year-month-day hierarchical structure

The subtree iterator HierarchyIterator is a forward iterator with its own state (m_block , m_idx , the stack m_stack). Moving to the next element is done by the advance() method, which traverses the subtree in DFS order until it finds a non-empty leaf or exhausts the stack.

Thanks to the standard `iterator_category` and type aliases, the iterator can be passed directly into the universal algorithm from level 1 without any modification.

3.3 Level 3

The table used. `Hash_Map_Data_Structure` is a wrapper over `ds::adt::HashTable<DateKey, AccidentBucket>` from the `libds` library. The table implementation successfully passes the published tests (`./libds.tests/ds-tests --test Tests.HashTable`).

The key, the date, is packed into a `uint32_t` via the function `make_date_key(date) = year·10000 + month·100 + day`. Choosing a compact integer key allows the hash function from `libds` to be used directly (modular hash over an integer) and avoids the need to compare complex objects on collisions.

Handling duplicates. The data typically contains multiple accidents with the same date. On insertion, therefore, the `insert(const Accident&)` method first tries to find the cell for the given key:

- If it exists, a new pointer is added to its accidents list. (Insertion into an `ImplicitList` is $O(1)$ at the end.)
- If it does not exist, a new `AccidentBucket` is created with a single pointer and inserted into the table.

This way no record is overwritten, no key-uniqueness check is applied (preserving the average $O(1)$ efficiency), and every accident remains findable.

Time complexity analysis of the `insert` and `find` operations.

Table design in libds. `ds::adt::HashTable` resolves collisions by chaining (separate chaining): the primary area is an array of m synonym lists, and a key falls into the list with index `hash(key) % m`. The number of primary slots is the constant `CAPACITY = 100`, and the table never rehashes; m stays 100 for the whole run. The load factor therefore grows linearly with the number of distinct keys, $\alpha = n_{\text{keys}}/m$.

Operation (general, m fixed)	Average case	Worst case
<code>find(date)</code>	$O(1 + n/m)$	$O(n)$
<code>insert(accident)</code>	$O(1 + n/m)$	$O(n)$

- `find` = $O(1 + \alpha)$. The hash is computed ($O(1)$) and the synonym list of the relevant slot, with average length $\alpha = n/m$, is traversed linearly. Since m is a constant, this is asymptotically $O(n)$ in the number of distinct keys.
- `insert` = $O(1 + \alpha)$. `UnsortedExplicitSequenceTable::insert` first verifies the absence of the key (contains, which is the same linear list traversal as `find`) and only then inserts the element at the front in $O(1)$. Insertion therefore inherits the linear complexity of the lookup, again $O(n)$ as the number of distinct keys grows.
- The worst case $O(n)$ occurs with a degenerate hash, where all keys map to a single slot and one synonym list has length n .

6UI0004 Algorithms and Data Structures 1

Empirical measurement. We measured the behaviour with a standalone benchmark (complexities/hash_map_analyzer): for each size from 10,000 to 150,000 distinct keys we ran 200 runs and averaged the time of a single operation.

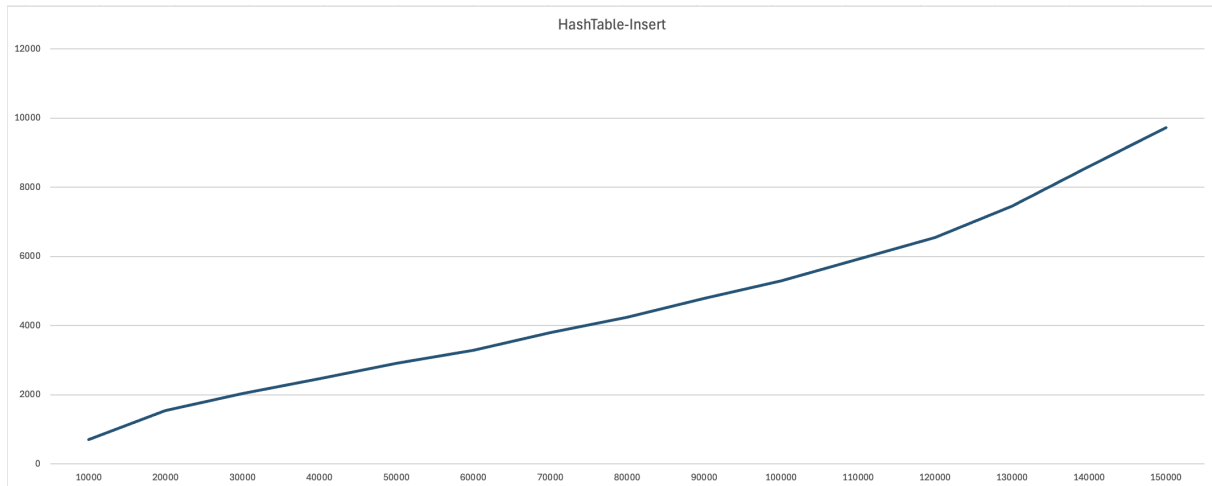


Figure 6: Measured times of the insert operation

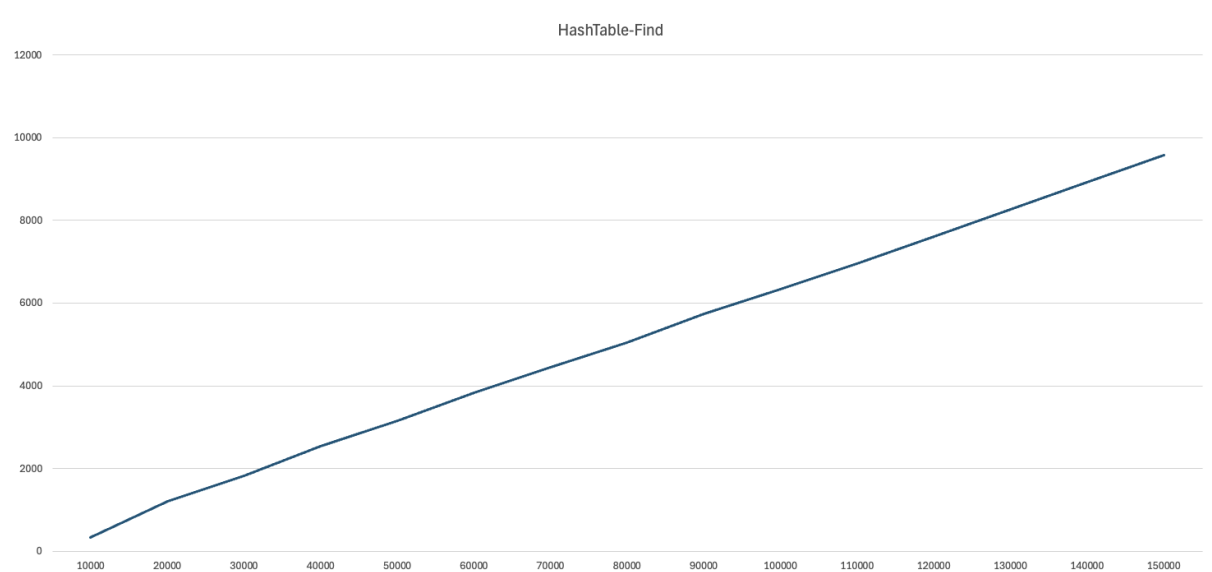


Figure 7: Measured times of the find operation

The measured averages confirm the linear trend predicted by theory:

Number of keys	10,000	50,000	100,000	150,000	Growth
insert [ns]	692	3,172	5,573	10,186	$14.7\times$
find [ns]	389	3,024	6,880	12,374	$31.8\times$

With a 15-fold increase in the number of keys, the time of both operations grew roughly 15–32-fold, i.e. linearly, exactly as follows from $O(1 + n/m)$ with constant m . (A table with

capacity doubling would keep α below a constant and the curve would stay flat; the libds version does not have this optimization.)

Consequence for this application. The key is the date, so the number of distinct keys is bounded by the calendar — for the years 2023–2025 it is at most $\sim 1,095$ values, independent of the number of accidents (which can be in the millions). With $m = 100$, $\alpha \approx 11$ and stays constant; further accidents on the same day only lengthen the list in the relevant cell (insertLast in $O(1)$), they do not add a new key. The date_table lookup is therefore practically $O(1)$ for real data, and the fixed capacity is not a limitation here; the linear growth in the charts occurs only in the artificial scenario with a growing number of unique keys (the benchmark).

Impact on loading. The table is filled once, in a single pass over the DataFrame after the hierarchy has been built. With a bounded number of date keys each insert is $O(1)$ and the whole fill is $O(n)$, so adding the table does not change the linear complexity of loading from level 2.

3.4 Level 4

The sorting algorithm. `data_sorter<T>::sort` is a custom implementation of a recursive top-down MergeSort with a deterministic $O(n \log n)$ complexity. It sorts a `std::vector<const Accident*>` obtained by the universal algorithm from level 1 over the subhierarchy of the current node. During merging only pointers (8 bytes) are copied, not whole records.

Sorting over the subhierarchy of the current node from level 2 (`Cli_Command_Logic::sort_from_tree`) can be run with two comparators:

- `compareDateTime`: the primary key is `Accident::date`, the secondary `Accident::time`. The lambda function has the signature `bool(const Accident*, const Accident*)` and returns true if the first record has an earlier date/time.
- `compareVehiclesAndCasualties`: the primary key is `number_of_vehicles`, the secondary `number_of_casualties`.

Both comparators are defined as lambda functions in the private part of `Cli_Command_Logic`. The algorithm itself takes them via `std::function`, which makes it possible to add another comparator (e.g. by severity) without any modification to the algorithm body.

Printing the sorted structure. After sorting, `DataFrame::execute(temp.begin(), temp.end(), predTrue, printWithKeys)` is called over the sorted `std::vector`. The `printWithKeys` processor prefixes each record with the pair of values it was sorted by (e.g. `[2024/05/12 06:30] ...` or `[V:3 C:2] ...`). This satisfies the assignment's requirement that the data be printed together with the values by which they were sorted.

4 User guide

The application is a textual interactive console. It is started by building the `semestral_project` target (e.g. in build):

```
./semestral_project
```

After launch the CSV files given as command-line arguments are loaded automatically (if any were provided). The main loop reads one line from stdin and treats the first word as the command.

4.1 Command overview

Command	Description	Example
help	Prints the available commands.	help
flags	Prints the available filter flags.	flags
data_meaning	Prints the mapping of enumeration values.	data_meaning
load <path>	Loads a CSV file.	load data/collisions-2023.csv
filter <flags>	Sets the predicate from filter flags.	filter -w 7 -j 2 -v 3 *
print	Runs the algorithm with the current predicate and prints the matches.	print
print -d	Prints the last saved snapshot (save_data).	print -d
date <interval>	Sets the predicate to a time range.	date -T 15/05/2024 06:00 15/05/2024 10:00
save_data <flags>	Runs the algorithm with procInsertInto, stores matches into m_filtered_data and prints them.	save_data -T 12/03/2023 -w 1 -a 1 -j 3
date_table <DD/MM/YYYY>	Looks up records in the hash table for the given day.	date_table 12/10/2023
tree ls	Prints the sons of the current hierarchy node.	tree ls
tree down <i>	Descends to the son with index i.	tree down 0
tree up	Ascends to the parent.	tree up
tree root	Jumps to the hierarchy root.	tree root
tree print [flags]	Filters and prints records in the subhierarchy.	tree print -w 7
tree filter <flags>	Sets the predicate from filter flags and prints matches in the subhierarchy of the current node.	tree filter -T 7:00 8:00

Command	Description	Example
tree save_data [flags]	Stores matches from the subhierarchy into m_filtered_data.	tree save_data -j 2
tree sort 1 [flags]	Sorts the subhierarchy by date and time.	tree sort 1 -w 7
tree sort 2 [flags]	Sorts by the number of vehicles and casualties.	tree sort 2
exit	Exits the application (frees memory).	exit

4.2 Filter flags

Filter flags serve as input for the predConditions predicate. Their complete list:

Flag	Meaning	Example
-co N	Collision year.	-co 2024
-l N	Speed limit.	-l 30
-v MIN [MAX]	Number of vehicles (range; * for unbounded).	-v 2 5 · -v * 5
-c MIN [MAX]	Number of casualties (range; * for unbounded).	-c 1 3 · -c 1 *
-w N	Weather (1–9, data_meaning shows the meaning).	-w 7 (fog)
-j N	Junction.	-j 2 (T-junction)
-a N	Severity.	-a 1 (fatal)
-p N	Pedestrian crossing.	-p 13 (zebra)
-d N	Day of week (1=Sunday, 7=Saturday).	-d 6
-T <date> [time] [date] [time]	Date-time interval. The date is optional: if only a time (HH:MM) is given, the time window applies to every day.	-T 7:00 8:00 · -T 15/05/2024 16:00 20/05/2024 20:00
reset / -df	Resets the predicate to “match all”.	filter reset

An asterisk * (or omitting the upper/lower end) marks an unbounded side of the interval for numeric flags. With -T the time can be omitted (default 00:00–23:59) and a single day given; likewise the date can be omitted and only a time given (e.g. -T 7:00 8:00) — the time window then applies to every day, which is useful in the tree, where the year and month are determined by the current node.

4.3 Typical scenario

1. load data/dft-road-casualty-statistics-collision-2024.csv: load the data.
2. tree root -> tree ls -> tree down 0 (year) -> tree ls -> tree down 4 (5th month): navigate the hierarchy.
3. tree filter -T 7:00 8:00: in the current node (May 2024) print only the accidents that happened between 07:00 and 08:00 (time window without a date).
4. tree sort 1 -w 7 -j 2: sort the accidents during fog at T-junctions in May 2024 by date and time, prefixing each record with a timestamp.
5. date_table 12/10/2023: instant lookup of accidents from an exact day via the table.

4.4 Exiting

The exit command correctly frees all memory. The application is linked with heap-monitor, so no memory leaks remain on a regular exit.

5 Programmer's guide

This section describes how to extend the application with new functionality without having to touch the body of the universal algorithm.

5.1 Adding a new predicate

A predicate is a `std::function<bool(const Accident*)>`. It is enough to write it as a lambda function in `lib/cli/cli_commands_logic.hpp` in the private section and, in `Cli_Command_Logic`, create a new command or flag that stores it into `m_predicate` after setting its parameters.

```
const std::function<bool(const Accident*)> predHasCasualty =  
    [](const Accident& a) { return a.number_of_casualties > 0; };
```

Using the new predicate requires no change in `DataFrame::execute`: the algorithm works through `std::function<bool(const T*)>`, so it accepts any compatible callable.

5.2 Adding a new processor

A processor is a `std::function<void(const Accident*)>` (or, with context, `void(Container&, const Accident&)` if it needs the target structure). The project has two basic processors:

- `procPrint` in `Cli_Helper_Printer`: formatted output,
- `procInsertInto` in `Cli_Command_Logic`: insertion into a `std::vector<const Accident*>`.

Adding a new processor (e.g. writing to a `std::ofstream`):

```
std::ofstream out("filtered.csv");
auto procToCsv = [&out](const Accident& a) {
    out << static_cast<int>(a.date.year()) << ','
        << static_cast<unsigned>(a.date.month()) << ','
        << static_cast<unsigned>(a.date.day()) << '\n';
};
m_data_frame.execute(m_data_frame.begin(), m_data_frame.end(),
                    m_predicate, procToCsv);
```

5.3 Adding a new comparator for sorting

A comparator for `data_sorter<const Accident*>` has the signature `bool(const Accident*, const Accident*)`. The algorithm takes it via `std::function`, so it is enough to write a new lambda function in `Cli_Command_Logic` and wire it up in `sort_from_tree`. Example: sorting by severity, secondarily by the number of casualties:

```
const std::function<bool(const Accident*, const Accident*)> compareSeverity =
    [](const Accident* a, const Accident* b) {
        if (a->collision_severity != b->collision_severity)
            return static_cast<int>(a->collision_severity) <
                static_cast<int>(b->collision_severity);
        return a->number_of_casualties > b->number_of_casualties;
    };
```

The body of `data_sorter<T>::sort` does not need to be modified.

5.4 Adding a new type of hierarchy traversal

`HierarchyIterator` is a forward iterator that traverses the leaves in DFS pre-order. If a different kind of traversal were needed (e.g. BFS, or only the sons without recursing into the subtree), a custom iterator must be implemented that satisfies the same interface (the type aliases `value_type`, `reference`, `pointer`, `difference_type`, `iterator_category`, and the operators `*`, `++`, `==`, `!=`). Thanks to the template parameter `Iterator` in `DataFrame::execute`, it can then be used without changing the algorithm.

5.5 Extending the Accident record

If a new field had to be added (e.g. district, map coordinates), the steps are:

1. Add a data member to struct `Accident` in `lib/structs/accident.hpp`.
2. In `CSV_Loader<T>::load_csv_data` add the column name to the `map`.



6UI0004 Algorithms and Data Structures 1

3. Add the corresponding `handle_*` method (use the existing `handle_weather_conditions` etc. as a template).
4. Optionally add an enum (if the field is categorical) and a flag to `Cli_Helper_Printer::FILTER_FLAGS`.

Thanks to the template design of `DataFrame<T>`, `data_sorter<T>` and `CSV_Loader<T>`, no changes are needed in these classes.